

Advanced Server Side Scripting

Unit-3

OOP In PHP

Laxman Pr Pant

Object and class in PHP

CLASS:

- A class is a description of an objects property and behaviors.
- Creating class as good as creating data type.
- Class is defining a category of data.
- **Class** is a **blueprint** of **related data members(properties/variable)** and **methods(function)**.
- **Class** is a self contained independent **collection** of **variables** and **functions**, which work together to perform one or more specific related task.

Syntax:

```
<access specifier>class <class-name>
{
    Property and methods.
}
```

Example:

```
class demo{  
    function add()  
    {  
        $x=800;  
        $y=200;  
        $sum=$x+$y;  
        echo "sum of given no=".$sum."<br/>";  
    }  
    function sub()  
    {  
        $x=1000;  
        $y=200;  
        $sub=$x-$y;  
        echo "Sub of given no=".$sub;  
    }  
}  
?>
```

Object:

- Object is an real world entity
- Object is an instance of class.
- Object consumes memory to hold property.
- Once a class has been defined, **objects** can be created from the class through “**new**” keyword.
- Class **methods** and **properties** can directly be **accessed** through this **object** instance.
- Connector (arrow operator->) symbol used to **connect objects** to their **properties** or **methods**.

Syntax

\$<object name>=new <class-name>();

\$<objectname>->property or method();

Example:

```
<?php
class demo
{
function add($a,$b)
{
$sum=$a+$b;
echo "Sum=".$sum."<br/>";
}
function sub($x,$y)
{
$sub=$x-$y;
echo "Subtraction=".$sub;
}
}
$obj= new demo();
$obj->add(800,200);
$obj->sub(1000,500);
?>
```

Example:

```
<?Php
class person
{
    public $name;
    function show()
    {
        echo "Your Name:". $this->name;
        echo "<br>";
    }
}
$p1=new person();
$p1->name="Laxman Pant";
$p1->show();
    $p1->show();

?>
```

Defining , Using properties and Method

- **Method:** In Object Oriented Programming in PHP, methods are functions inside classes. Their declaration and behavior are almost similar to normal functions by using function keyword, except their special uses inside the class.

Example:

```
<?php
class A
{
function addNumber()
{
    $x=6;
    $y=8;
    $sum=$x+$y;
    echo $sum;
}

$obj= new A();
$obj->addNumber();

?>
```

Constructor and destructor in PHP

- If a class name and function name will be similar in that case function is known as constructor.
- Constructor is special type of method because its name is similar to class name.
- Constructor automatically calls when object will be initializing.

Example:

```
<?php
class A
{
function testA()
{
echo "This is test method of class A";
}
function A()
{
echo "This is user defined constructor of class A"."<br/>";
}
}
$obj= new A();
$obj->testA();
?>
```


Predefine Constructor

- PHP introduce a new functionality to define a constructor i.e __construct().
- By using this function we can define a constructor.
- It is known as predefined constructor. Its better than user defined constructor because if we change class name then user defined constructor treated as normal method.
- **Note:**if predefined constructor and user defined constructor, both define in the same class, then predefined constructor treat like a Constructor while user defined constructor treated as normal method.

```
<?php  
  
class A{  
  
    function A()  
    {  
        echo "user defined constructor";  
    }  
    function __construct()  
    {  
        echo "This is predefined constructor";  
    }  
}  
$obj= new A();  
?>
```

For Example:

```
class person
{
    public $name;
    public $age;

    function __construct($x,$y)
    {
        $this->name=$x;
        $this->age=$y;
    }
    function show()
    {
        echo $this->name ."-". $this->age;
    }
}

$obj1=new person("Lalit Kumar",21);
$obj1->show();

?>
```

Destructor in PHP

- The Destructor method will be called as soon as there are no other references to a particular object, or in any order during the shutdown sequence.
- Destructor automatically call at last.
- **Note :** __destruct() is used to define destructor.

Example:

```
<?php
class demo
{
function __construct()
{
echo "object is initializing their propertie"."<br/>";
}
function work()
{
echo "Now works is going "."<br/>";
}
function __destruct()
{
echo "after completion the work, object destroyed automatically";
}
}
$obj= new demo();
$obj->work();
//to check object is destroyed or not
echo is_object($obj);
?>
```

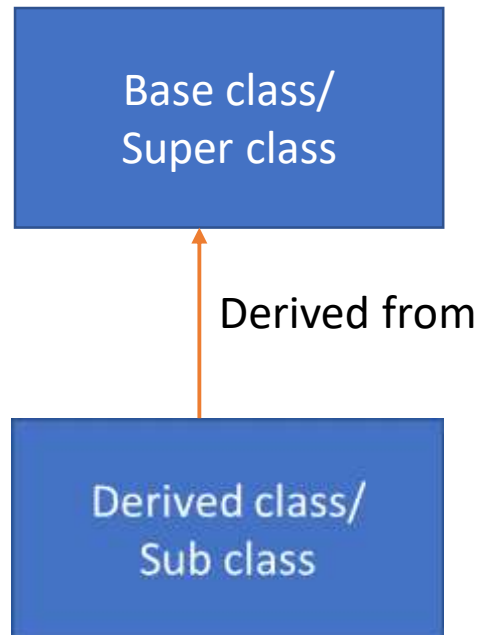
Best Example of constructor and destructor.

```
<?Php
```

```
class person {  
    public $name;  
    public $age;  
    function __construct($x,$y) {  
        $this->name=$x;  
        $this->age=$y;  
    }  
    function show(){  
        echo $this->name . "-". $this->age. "<br/>";  
    }  
    function __destruct(){  
        echo "Thank you";  
    }  
}  
  
$obj1=new person("Lalit Kumar",21);  
$obj1->show();  
?>
```

Inheritance in PHP

- It is the capability of one class to inherit properties from another class.
- The technique of building new classes from the existing class is called inheritance
- To implement inheritance in php using keyword **extends**



Base class:

- It is class whose properties are inherited by another class.
- It is also called **super class**

Derived class:

- It is the class that inherit properties from base class.
- It is also called **sub class**.
- It inherits all the properties of the base class and can add additional features to the derived class.

Types of Inheritance

There are mainly three types of inheritance in php.

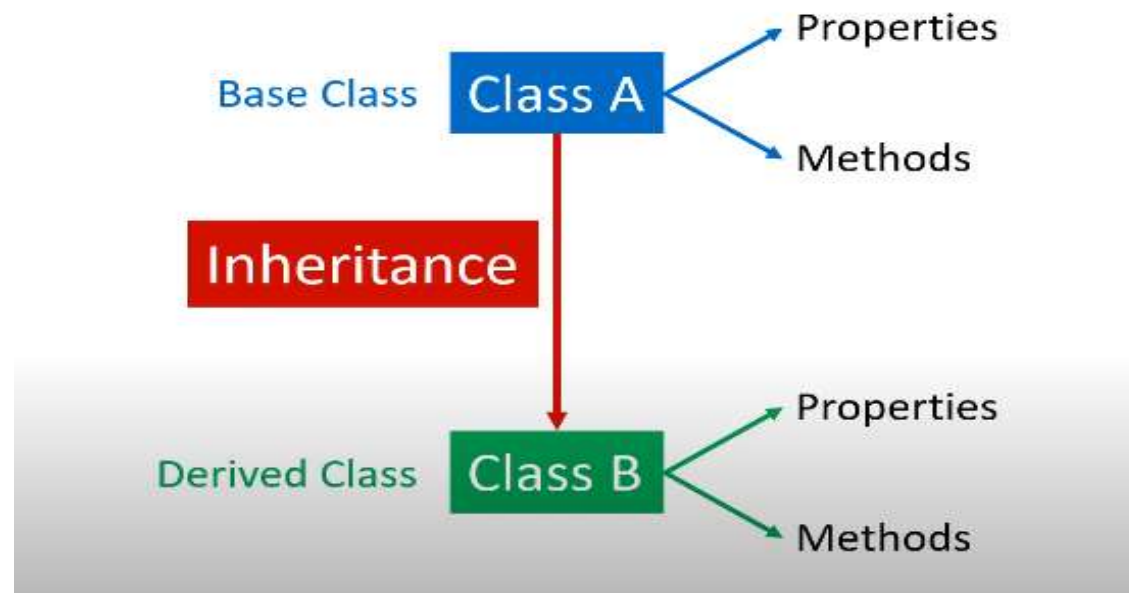
- Single inheritance
- Multi-level inheritance
- Hierarchical inheritance

Single inheritance:

- If a class is derived from one base class, it is called single inheritance.

Syntax:

```
class A    [It's Base class]
{
    members of class A.
}
class B extends A    [It's child class]
{
    members of class B.
}
```



```
<?php
```

```
class student
```

```
{
```

```
    public $roll;
```

```
    public $name;
```

```
    function stud()
```

```
    {        $this->roll=8;
```

```
        $this->name="Laxman";
```

```
    }
```

```
}
```



```
class marks extends student  
{  
  
    function mark()  
    {  
  
        $this->sub1=78;  
  
        $this->sub2=90;  
  
    }  
  
    function show()  
    {  
  
        echo "Your Name is:". $this->name;  
  
        echo "<br>";  
  
        echo "Your Roll is:". $this->roll;  
  
        echo "<br>";  
  
        echo "Your English Marks is:". $this->sub1;  
  
        echo "<br>";  
  
        echo "Your Math Marks is:". $this->sub2;  
  
    }  
  
}
```

```
$obj1=new marks();
```

```
$obj1->stud();
```

```
$obj1->mark();
```

```
$obj1->show();
```

```
?>
```

Multi-Level Inheritance:

In this inheritance, a derived class is created from another derived class

Syntax:

Class base

{

members of class base.

}

Class derived1 extends base

{

members of class derived.

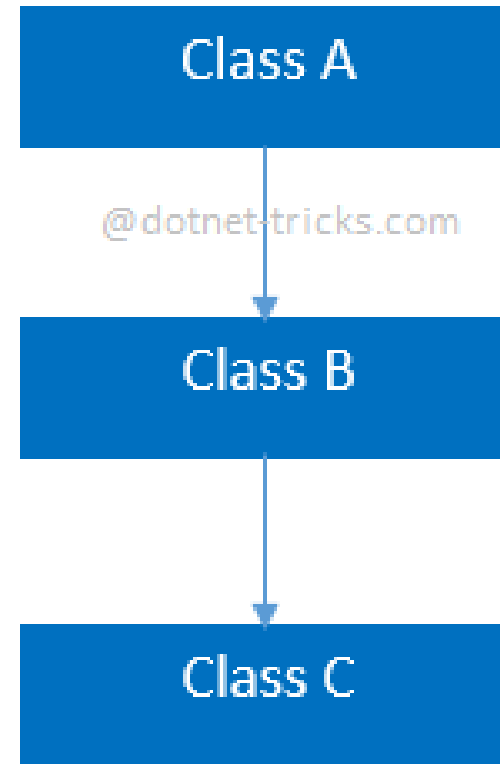
}

Class derived2 extends derived1

{

members of class derived.

}



Multi-Level Inheritance

```
<?php
```

```
class student
```

```
{
```

```
    public $roll;
```

```
    public $name;
```

```
    function stud()
```

```
    {        $this->roll=8;
```

```
            $this->name="Laxman";
```

```
    }
```

```
}
```

```
class marks extends student
```

```
{
```

```
    function mark()
```

```
    {
```

```
        $this->sub1=78;
```

```
        $this->sub2=90;
```

```
    }
```

```
}
```

```
class address extends marks
```

```
{
```

```
    function details()
```

```
    {
```

```
        $this->address="Kathmandu";
```

```
    }
```

```
    function show()
```

```
    {
```

```
        echo "Your Name is:".$this->name;
```

```
        echo "<br>";
```

```
        echo "Your Roll is:".$this->roll;
```

```
        echo "<br>";
```

```
        echo "Your English Marks is:".$this->sub1;
```

```
        echo "<br>";
```

```
        echo "Your Math Marks is:".$this->sub2;
```

```
        echo "<br>";
```

```
        echo "The Address is:".$this->address;
```

```
    }
```

```
}
```

```
$obj1=new address();
```

```
$obj1->stud();
```

```
$obj1->mark();
```

```
$obj1->details();
```

```
$obj1->show();
```

```
?>
```

Hierarchical Inheritance

In this inheritance, more than one derived classes are created from a single base class and further child classes act as parent classes for more than one child classes.

Syntax:

Class base

{

members of class base.

}

Class derived1 extends base

{

members of class derived.

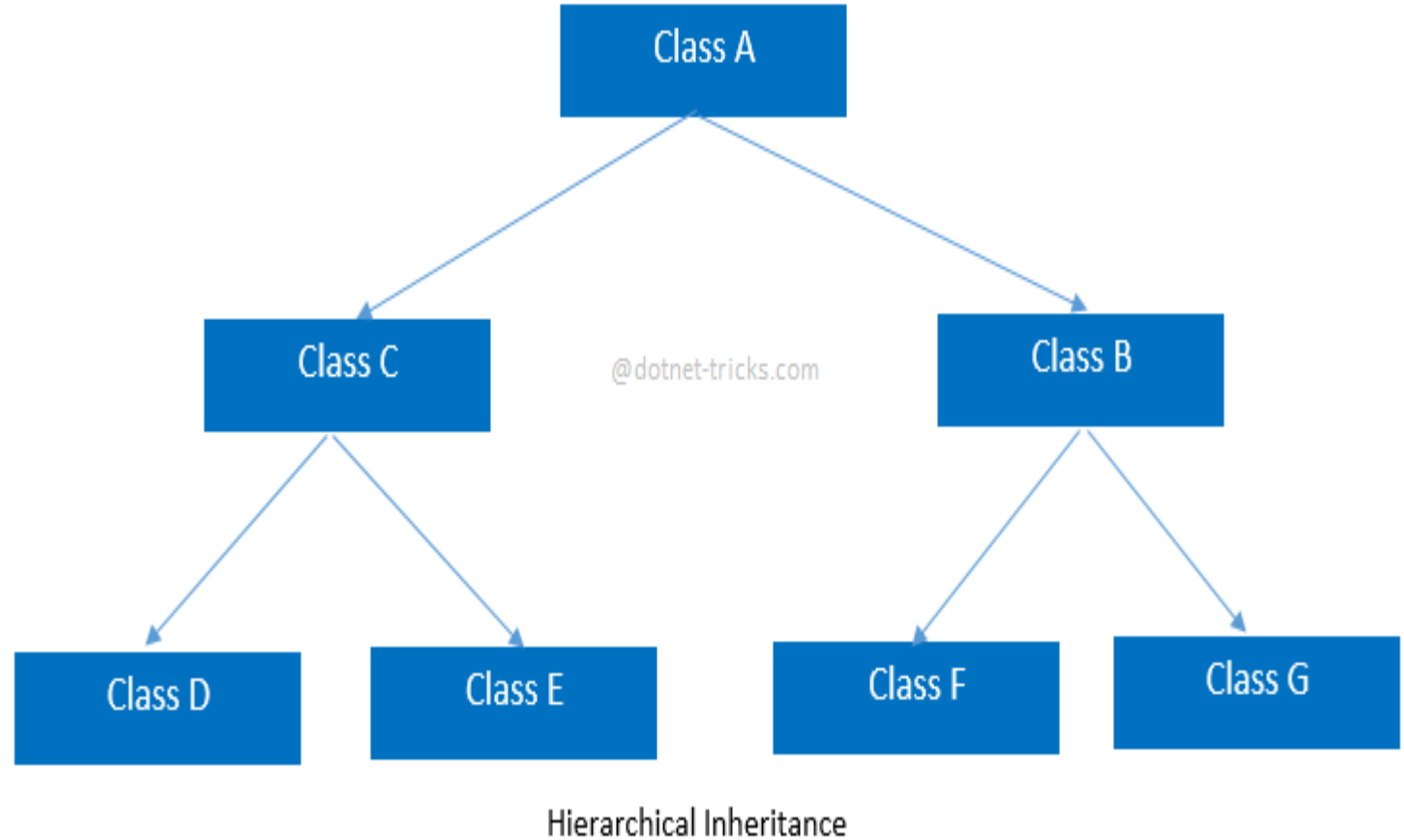
}

Class derived2 extends base

{

members of class derived.

}



Method Overriding and Properties in PHP

- Method overriding allows a child class to provide a specific implementation of a method already provided by its parent class. To override a method, we redefine that method in the child class with the same name, parameters, and return type.
- The method in the parent class is called **overridden method**, while the method in the child class is known as the **overriding method**. The code in the overriding method overrides (or replaces) the code in the overridden method.

Example of Method and properties overriding:-

```
<?php
class student{
public $name="Laxman";
public $roll=5;
public $address="KTM";
function show()
{
    echo "Your Name is:".$this->name;
    echo"<br>";
    echo "Your Name is:".$this->roll;
    echo"<br>";
    echo "Your Name is:".$this->address;
    echo"<br><hr>";
}
}
```

```
class address extends student{
public $name="Kumar";
public $roll=6;
public $address="Lalitpur";
function show()
{
echo "Your Name is:".$this->name;
echo"<br>";
echo "Your Name is:".$this->roll;
echo"<br>";
echo "Your Name is:".$this->address;
}
}
$obj1=new student();
$obj1->show();
$obj2=new address();
$obj2->show();
?>
```

Encapsulation in PHP

- The wrapping up of data and methods into a single unit (called class) is known as encapsulation. Encapsulation is a protection mechanism for the data members and methods present inside the class. In the encapsulation technique, we are restricting the data members from access to outside world end-user.
- In PHP, encapsulation is utilized to make the code more secure and robust. Using encapsulation, we are hiding the real implementation of data from the user and also does not allow anyone to manipulate data members except by calling the desired operation.

in a capsule.



[All same type of variables are encapsulated

Example of Encapsulation in PHP

```
<?php
class car{
    protected $model="Maruti";
    public function getModel()
    {
        echo $this->model;
    }
}
class sportcar extends car{
    public function getModel()
    {
        echo $this->model;
    }
}
$obj1=new sportcar();
$obj1->getModel();
?>
```

Polymorphism in PHP

- Polymorphism in OOPs is a concept that allows you to create classes with different functionalities in a single interface.
- Generally, it is of two types: compile-time (overloading) and run time (overriding), but polymorphism in PHP does not support overloading, or in other words, compile-time polymorphism. The most significant benefit of polymorphism in PHP is that you don't have to worry about the technicalities of defining which code is written in which class, as all will be used in the same way.

```

<?php
interface Shape{
    public function calArea();
}
class Circle implements Shape{
    private $radius;
    public function __construct($radius)
    {
        $this->radius=$radius;
    }
    public function calArea()
    {
        echo $this->radius*$this->radius*pi();
    }
}

```

```

class Rectangel implements Shape{
    private $width;
    private $height;
    public function __construct($width,$height)
    {
        $this->width=$width;
        $this->width*$this->height;
    }
}
$circ=new Circle(3);
$rect=new Rectangle(3,4);
echo $circ->calcArea();
echo "<br>";
echo $rect->calcArea();

?>

```

Static Member in PHP

- Static members declared with the **static** keyword in a class. Static variables should be declared outside any block, means should not be inside any functions, but should just inside class.
- With out object ,we can access the properties and method in a class . For calling the properties and method the **\$self::** for self class and if inherited **parent::** keyword is used instead of **\$this->** .

Static Members In PHP

```
<?php
class car{
    public static $model="Maruti";
    public static function getModel()
    {
        echo self::$model;
    }
}

class sportcar extends car{
    public static function getModel()
    {
        echo parent::$model;
    }
}

$obj1=new sportcar();
$obj1->getModel();
?>
```

Exception handling in PHP

- An *exception* is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.
- Exception handling is the process of responding to unwanted or unexpected events when a computer program runs. Exception handling deals with these events to avoid the program or system crashing, and without this process, exceptions would disrupt the normal operation of a program.

We will show different error handling methods:-

- Basic use of Exceptions
- Creating a custom exception handler
- Multiple exceptions
- Re-throwing an exception
- Setting a top level exception handler

Why Exception handling

- PHP provides a powerful mechanism, exception handling. It allows you to handle runtime errors such as IOException, SQLException, ClassNotFoundException, and more. A most popular example of exception handling is - divide by zero exception, which is an arithmetic exception.
- Exception handling is almost similar in all programming languages. It changes the normal flow of the program when a specified error condition occurs, and this condition is known as exception. PHP offers the following keywords for this purpose:

Exception Handling methods

1)try -

- The try block contains the code that may have an exception or where an exception can arise. When an exception occurs inside the try block during runtime of code, it is caught and resolved in catch block. The try block must be followed by catch or finally block. A try block can be followed by minimum one and maximum any number of catch blocks.

2)catch -

- The catch block contains the code that executes when a specified exception is thrown. It is always used with a try block, not alone. When an exception occurs, PHP finds the matching catch block.

3)throw -

- It is a keyword used to throw an exception. It also helps to list all the exceptions that a function throws but does not handle itself.

?php

```
class DivideByZeroException extends Exception { }  
class DivideByNegativeNoException extends Exception { }  
function checkdivisor($dividend, $divisor){  
    // Throw exception if divisor is zero  
    try {  
        if ($divisor == 0) {  
            throw new DivideByZeroException;  
        }  
        else if ($divisor < 0) {  
            throw new DivideByNegativeNoException;  
        }  
        else {  
            $result = $dividend / $divisor;  
            echo "Result of division = $result </br>";  
        }  
    }  
}
```

```
catch (DivideByZeroException $dze) {  
    echo "Divide by Zero Exception! </br>";  
}  
catch (DivideByNegativeNoException $dnne) {  
    echo "Divide by Negative Number Exception </br>";  
}  
catch (Exception $ex) {  
    echo "Unknown Exception";  
}  
}
```

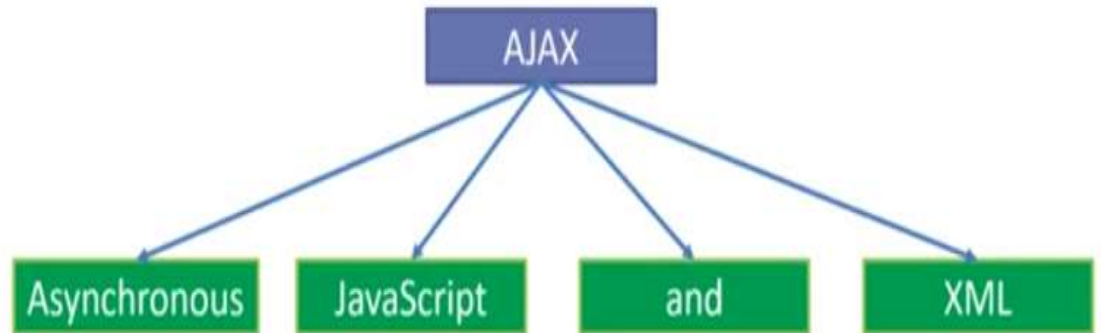
```
checkdivisor(18, 3);  
checkdivisor(34, -6);  
checkdivisor(27, 0);
```

?>

```
Result of division = 6  
Divide by Zero Exception!  
Divide by Negative Number Exception!
```

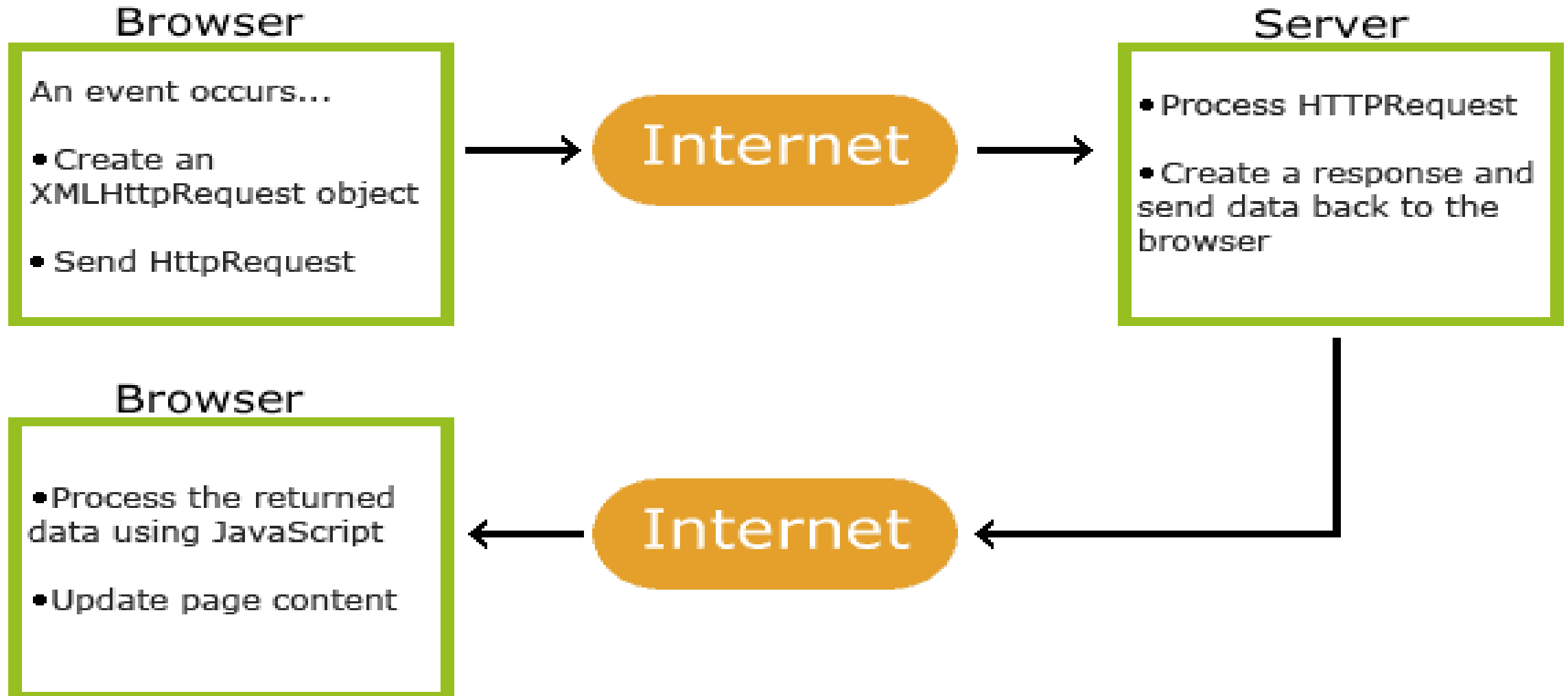
Overview of AJAX

- AJAX is Asynchronous JavaScript and XML
- AJAX allows web pages to be updated asynchronously by exchanging small amounts of data with web server behind the scene.
- This means that it is possible to update parts of a web page, without reloading the whole page.



AJAX is a technique for creating fast and dynamic web pages.

How AJAX Works



AJAX Components

The key technical component of AJAX are:

- XHTML – a stricter, cleaner rendering of HTML into XML.
- CSS for marking up and adding styles.
- The JavaScript Document Object Model (DOM) which allows the content, structure and style of a document to be dynamically accessed and updated.
- The XMLHttpRequest object which exchanges data asynchronously with the Web server reducing the need to continually fetch resources from the server.

Asynchronous Data Transfer with XMLHttpRequest

- All modern browser support the XMLHttpRequest object.
- The XMLHttpRequest object is used to exchange data with a server behind the scenes.
- `Req=new XMLHttpRequest();`
- Sending request to the server
`Req.open("GET","abc.php", true);`
`Req.send();`

`Req.open("GET","abc.php?x=25", true);`
`Req.send();`

XMLHttpRequest Object Methods

Method	Description
<code>new XMLHttpRequest()</code>	Creates a new XMLHttpRequest object
<code>abort()</code>	Cancels the current request
<code>getAllResponseHeaders()</code>	Returns header information
<code>getResponseHeader()</code>	Returns specific header information
<code>open(<i>method</i>, <i>url</i>, <i>async</i>, <i>user</i>, <i>psw</i>)</code>	Specifies the request <i>method</i> : the request type GET or POST <i>url</i> : the file location <i>async</i> : true (asynchronous) or false (synchronous) <i>user</i> : optional user name <i>psw</i> : optional password
<code>send()</code>	Sends the request to the server Used for GET requests
<code>send(<i>string</i>)</code>	Sends the request to the server. Used for POST requests

XMI HttpRequest Object Properties

Property	Description
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
responseText	Returns the response data as a string
responseXML	Returns the response data as XML data
status	Returns the status-number of a request 200: "OK" 403: "Forbidden" 404: "Not Found" For a complete list go to the Http Messages Reference
statusText	Returns the status-text (e.g. "OK" or "Not Found")

onreadystatechange

- When a request to a server is sent. We want to perform some actions based on the response.
- The onreadystatechange event is triggered every time the readyState changes.
- The onreadystatechange stores a function to be called automatically each time the readyState property change.

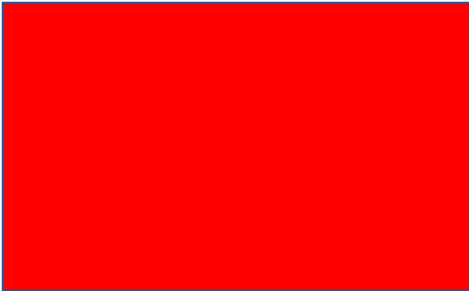
code

```
JS{
    var req= new XMLHttpRequest();
    req.open("GET","abc.php", true);
    req.send();
    req.onreadystatechange=function()
    {
        if(req.readyState==4 && req.status==200)
        {
            document.getElementById("myDiv").innerHTML;
            ML= req.responseText;
        }
    }
}
```

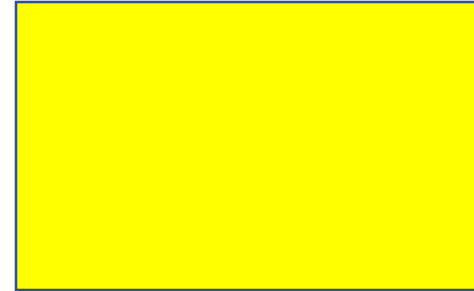
Example

- Fetching city name from server on selection of state.

Form.php



Cities.php



Formstyle.css



Formscript.js



```
<!DOCTYPE html>

<html>

<head>

  <title>Ajax Example</title>

  <link rel="stylesheet" type="text/css" href="formstyle.css">

  <script type="text/javascript" src="formscript.js"></script>

</head>

<body>

  <div id="form-div">

    <h1>Application form</h1>

    <form action="ajaxphp.php" method="post">

      <div>

        <label>Name</label>

        <input type="text" name="uname">

      </div>

      <div>

        <label>Select state</label>

        <select name="state" onchange="fetchcities(this.value)">

          <option>Select state </option>

          <option>Pradesh1</option>

          <option>Pradesh3</option>

          <option>Pradesh7</option>

        </select>

      </div>

    </form>

  </div>

</body>

</html>
```

```
</div>

<div>

  <label>Select City</label>

  <select id="cities">

    <option>First Select state </option>

  </select>

</div>

<div>

  <input type="submit" name="submit" value="submit">

</div>

</form>

</div>

</body>

</html>
```

Formstyle.css

#form-div

```
{  
    position: relative;  
    left: 30%;  
    width: 400px;  
    background-color: grey;  
    padding: 30px  
}
```

#form-div div

```
{  
    margin: 10px;  
}
```

input ,select{

```
    display: block;  
    padding: 10px;  
    width: 100%;  
    border-radius: 4px;  
}
```


Formscript.js

```
function fetchcities(str)
{
    //alert(str);
    var req=new XMLHttpRequest();
    req.open("get","http://localhost:81/Ajax1/cities.php?state="+str,true);
    req.send();
    req.onreadystatechange=function(){
        if (req.readyState==4 && req.status==200) {
            document.getElementById("cities").innerHTML=req.responseText;
        }

    };
}
```

```
<?php
    $state=$_GET['state'];
    $c1=array('jhapa','dhankuta');
    $c2=array('katmandu','lalitpur','Bhaktpur');
    $c3=array('dhangadi','mahendranagar');
    if ($state=='Pradesh1') {
        foreach ($c1 as $v) {
            echo "<option>$v</option>";
        }
    }
}
```

```
    else if ($state=='Pradesh3') {
        foreach ($c2 as $v) {
            echo "<option>$v</option>";
        }
    }
    else if($state=='Pradesh7')
    {
        foreach ($c3 as $v) {
            echo "<option>$v</option>";
        }
    }
    }else
    {
        echo "<option>select First State</option>";
    }
?>
```

Introduction to jQuery

- Developed by John Resign at Rochester institute of technology and released in January 2006.
- jQuery is small light weight java script library that simplifies programming with JavaScript.
- jQuery is fast, small and feature-rich JavaScript library.
- jQuery is cross-platform.
- jQuery means "write less do more"
- It makes things like HTML document traversal and manipulation, event handling, animation and AJAX.

Lightweight JavaScript Library

JavaScript

- Long Selector → Function() →
- Complex Animations → Function() →
- Lengthy DOM Manipulation → Function() →
- Lengthy Ajax Coding → Function() →



write less, do more

How to use or implement jQuery

Step 1: Download the jQuery.js file or library from jQuery.com.

- Step 2: Include jQuery.js file in HTML file
- Step 3: Do jQuery code in <script> tag
- The jQuery library is a single JavaScript file and reference it with the <script> tag.

```
<head>
```

```
<script src="jquery-3.4.1.min.js"></script>
```

```
</head>
```

OR

- It can also include from google CDN as

```
<head>
```

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
```

```
</head>
```

jQuery Syntax

- The jQuery syntax is tailor-made for **selecting** HTML elements and performing some **action** on the element(s).

Basic syntax is:

\$(selector).action()

A \$ sign to define/access jQuery

A (*selector*) to "query (or find)" HTML elements

A jQuery *action()* to be performed on the element(s)

Examples:

- `$(this).hide()` - hides the current element.
- `$("p").hide()` - hides all `<p>` elements.
- `$(".test").hide()` - hides all elements with `class="test"`.
- `$("#test").hide()` - hides the element with `id="test"`.

jQuery Selectors

- jQuery selectors allow you to select and manipulate HTML element(s).

element Selector

- The jQuery element selector selects elements based on the element name.
- Syntax: `$("element name")`
- Example: `$("p")`=selects all `<p>` elements in html document

#id Selector

- The jQuery *#id* selector uses the id attribute of an HTML tag to find the specific element.
- Syntax: `$("#id_value")`
- Example: `$("#test")`=selects element with `id="test"` in html document

.class Selector

- The jQuery *.class* selector finds elements with a specific class.
- Syntax: `$(".class_value")`
- Example: `$(".test")`=selects elements with `class="test"` in html document

***selector**

- Selects all elements
- Syntax : \$("*")

Some other selector:

\$(this)	Selects the current HTML element
\$("p.intro")	Selects all <p> elements with class="intro"
\$("p:first")	Selects the first <p> element
\$("ul li:first")	Selects the first element of the first
\$("ul li:first-child")	Selects the first element of every
\$("ul li:last-child")	Selects the last element of every

- Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>First jQuery Example</title>
  <script type="text/javascript"
src="https://ajax.googleapis.com/ajax/libs/jquery/2.1.3/jquery.min.js">
  </script>
  <script type="text/javascript" language="javascript">
$(document).ready(function() {
$("p").css("background-color", "pink");
});
</script>
</head>
<body>
<p>This is first paragraph.</p>
<p>This is second paragraph.</p>
<p>This is third paragraph.</p>
</body>
</html>
```

This is first paragraph.

This is second paragraph.

This is third paragraph.

jQuery Event

- jQuery events are the actions that can be detected by your web application.
- They are used to create dynamic web pages.
- An event shows the exact moment when something happens.
- These are some examples of events.
 - A mouse click
 - An HTML form submission
 - A web page loading
 - A keystroke on the keyboard
 - Scrolling of the web page etc.

Mouse Events

- click
- dblclick
- mouseenter
- mouseleave

Keyboard Events

- keyup
- keydown
- Keypress

Form Events

- submit
- change
- blur
- focus

Document/Window Events

- load
- unload
- scroll
- resize

Syntax for event methods

```
$("#p").click(function(){  
    // action goes here!!  
});
```

what should happen when the event fires

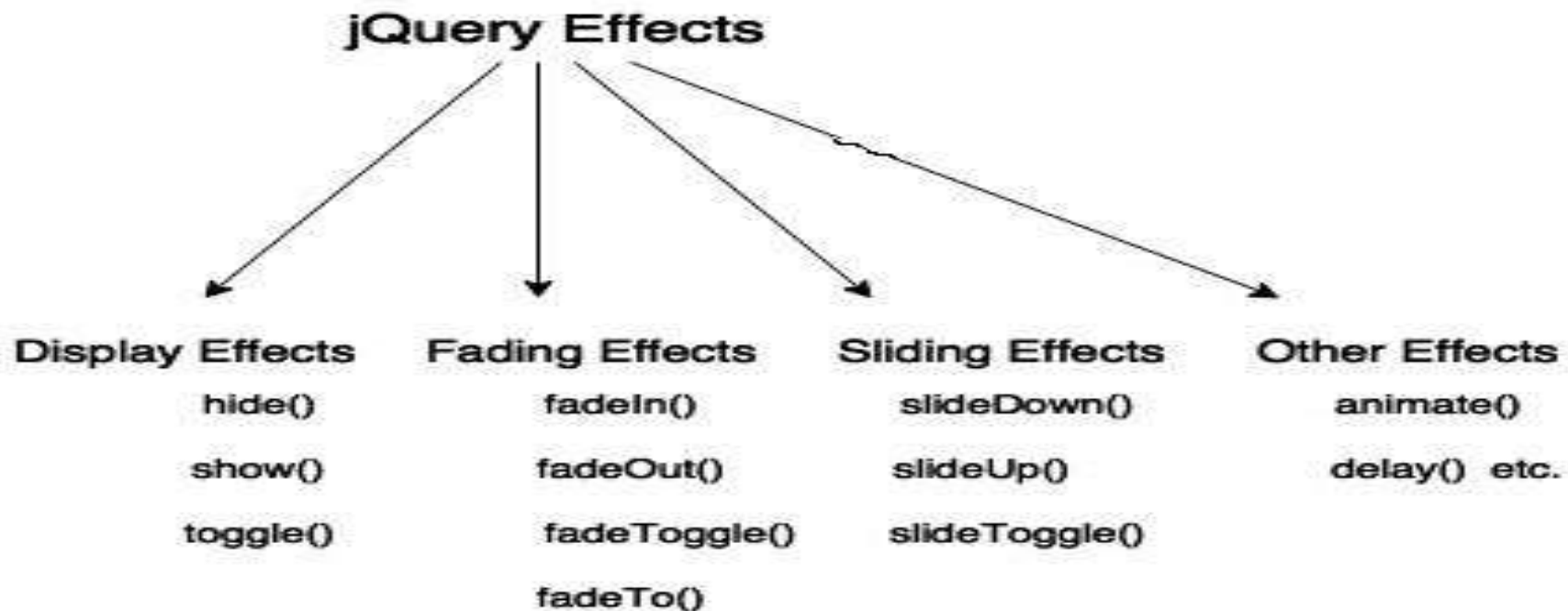
Example:

```
<!DOCTYPE html>  
<html>  
<head>  
<script  
src="https://ajax.googleapis.com/ajax/libs/jquery/1.11.2/j  
query.min.js"></script>  
<script>  
$(document).ready(function(){  
    $("#p").click(function(){  
        alert("This paragraph was clicked.");  
    });  
});  
</script>  
</head>  
<body>  
<p>Click on the statement.</p>  
</body>  
</html>
```

Click on the statement.

Jquery Effects

- jQuery enables us to add effects on a web page. jQuery effects can be categorized into fading, sliding, hiding/showing and animation effects.



Example (display effect)

```
<!DOCTYPE html>
<html>
<head>
<script src="jquery-3.3.1.min.js"></script>
<script>
$(document).ready(function(){
    $("#hide").click(function(){
        $("p").hide();
    });
    $("#show").click(function(){
        $("p").show();
    });
    $("#toggle").click(function(){
        $("p").toggle();
    });
});
</script>
</head>
```

```
<body>
<p>
<b>This is a little poem: </b><br/>
Twinkle, twinkle, little star<br/>
How I wonder what you are<br/>
Up above the world so high<br/>
Like a diamond in the sky<br/>
Twinkle, twinkle little star<br/>
How I wonder what you are
</p>
<table>
    <tr><td>
<button id="hide">Hide</button><td>
<td><button id="show">Show</button></td>
<td><button id="toggle">Toggle</button></td>
</tr>
</table>
</body>
</html>
```

Example(fading effect)

```
<!DOCTYPE html>
<html>
<head>
<script src="jquery-3.3.1.min.js"></script>
<script>
$(document).ready(function(){
    $("#b1").click(function(){
        $("#div1").fadeIn();
        $("#div2").fadeIn("slow");
    });
    $("#b2").click(function(){
        $("#div1").fadeOut();
        $("#div2").fadeOut("slow");
    });
    $("#b3").click(function(){
        $("#div1").fadeToggle();
        $("#div2").fadeToggle("slow");
    });
});
});
```

```
});
});
</script>
</head>
<body>
<p>See the fade effect example with different
parameters.</p>
<button id="b1">fade in </button>
<button id="b2"> fade out</button>
<button id="b3">fade toggle</button><br><br>
<div id="div1"
    style="width:80px;height:80px;display:none;bac
    kground-color:red;"></div>
<div id="div2"
    style="width:80px;height:80px;display:none;bac
    kground-color:green;"></div>

</body>
</html>
```

Example(slide effect)

```
<!DOCTYPE html>
<html>
<head>
<script src="jquery-3.3.1.min.js"></script>
<script>
$(document).ready(function(){
  $("#flip").click(function(){
    $("#panel").slideDown("slow");
  });
});
</script>
<style>
#panel, #flip {
  padding: 5px;
  text-align: center;
  background-color: #e5eccc;
  border: solid 1px #c3c3c3;
```

```
}
```

```
#panel {
  padding: 50px;
  display: none;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div id="flip">Click to slide down
  panel</div>
```

```
<div id="panel">Hello world!</div>
```

```
</body>
```

```
</html>
```


Jquery HTML

- jQuery contains powerful methods for changing and manipulating HTML elements and attributes.

Get Content - text(), html(), and val()

- Three simple, but useful, jQuery methods for DOM manipulation are:
- text() - Sets or returns the text content of selected elements
- html() - Sets or returns the content of selected elements (including HTML markup)
- val() - Sets or returns the value of form fields

Example

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js">
</script>
<script>
$(document).ready(function(){
  $("#btn1").click(function(){
    alert("Text: " + $("#test").text());
  });
  $("#btn2").click(function(){
    alert("HTML: " + $("#test").html());
  });
});
</script>
</head>
<body>

<p id="test">This is some <b>bold</b> text in a paragraph.</p>

<button id="btn1">Show Text</button>
<button id="btn2">Show HTML</button>

</body>
</html>
```

This is some **bold** text in a paragraph.

Show Text

Show HTML

Set Content - text(), html(), and val()

use the same three methods from the previous page to **set content**:

- **text()** - Sets or returns the text content of selected elements
- **html()** - Sets or returns the content of selected elements (including HTML markup)
- **val()** - Sets or returns the value of form fields

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<script src="jquery-3.3.1.min.js"></script>
```

```
<script>
```

```
$(document).ready(function(){
```

```
  $("#btn1").click(function(){
```

```
    $("#test1").text("Hello world!");
```

```
  });
```

```
  $("#btn2").click(function(){
```

```
    $("#test2").html("<b>Hello world!</b>");
```

```
  });
```

```
  $("#btn3").click(function(){
```

```
    $("#test3").val("Enetr input");
```

```
  });
```

```
});
```

```
</script>
```

```
</head>
```

```
<body>
```

```
<p id="test1">This is a paragraph.</p>
```

```
<p id="test2">This is another paragraph.</p>
```

```
<p>Input field: <input type="text" id="test3"  
  value="something"></p>
```

```
<button id="btn1">Set Text</button>
```

```
<button id="btn2">Set HTML</button>
```

```
<button id="btn3">Set Value</button>
```

```
</body>
```

```
</html>
```

Add New HTML Content

- There are four jQuery methods that are used to add new content:
 - `append()` - Inserts content at the end of the selected elements
 - `prepend()` - Inserts content at the beginning of the selected elements
 - `after()` - Inserts content after the selected elements
 - `before()` - Inserts content before the selected elements

Remove Elements/Content

- To remove elements and content, there are mainly two jQuery methods:
 - `remove()` - Removes the selected element (and its child elements)
 - `empty()` - Removes the child elements from the selected element

jQuery Manipulating CSS

- jQuery has several methods for CSS manipulation. We will look at the following methods:
 - **addClass()** - Adds one or more classes to the selected elements
 - **removeClass()** - Removes one or more classes from the selected elements
 - **toggleClass()** - Toggles between adding/removing classes from the selected elements
 - **css()** - Sets or returns the style attribute

Example:

```
<!DOCTYPE html>
<html>
<head>
<script src="jquery-3.3.1.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("#div1").addClass("important blue");
    });
});
</script>
<style>
.important {
    font-weight: bold;
    font-size: xx-large;
}
```

```
.blue {
    color: blue;
}
</style>
</head>
<body>

<div id="div1">This is some text.</div>
<div id="div2">This is some text.</div>
<br>

<button>Add classes to first div element</button>

</body>
</html>
```

Example(css() method)

```
<!DOCTYPE html>
<html>
<head>
<script src="jquery-3.3.1.min.js"></script>
<script>
$(document).ready(function(){
    $("button").click(function(){
        $("p").css({"background-color":
        "yellow", "font-size": "200%"});
    });
});
</script>
</head>
<body>

<h2>This is a heading</h2>
```

```
<p style="background-color:#ff0000">This
is a paragraph.</p>
<p style="background-color:#00ff00">This
is a paragraph.</p>
<p style="background-color:#0000ff">This
is a paragraph.</p>

<p>This is a paragraph.</p>

<button>Set multiple styles for p</button>

</body>
</html>
```

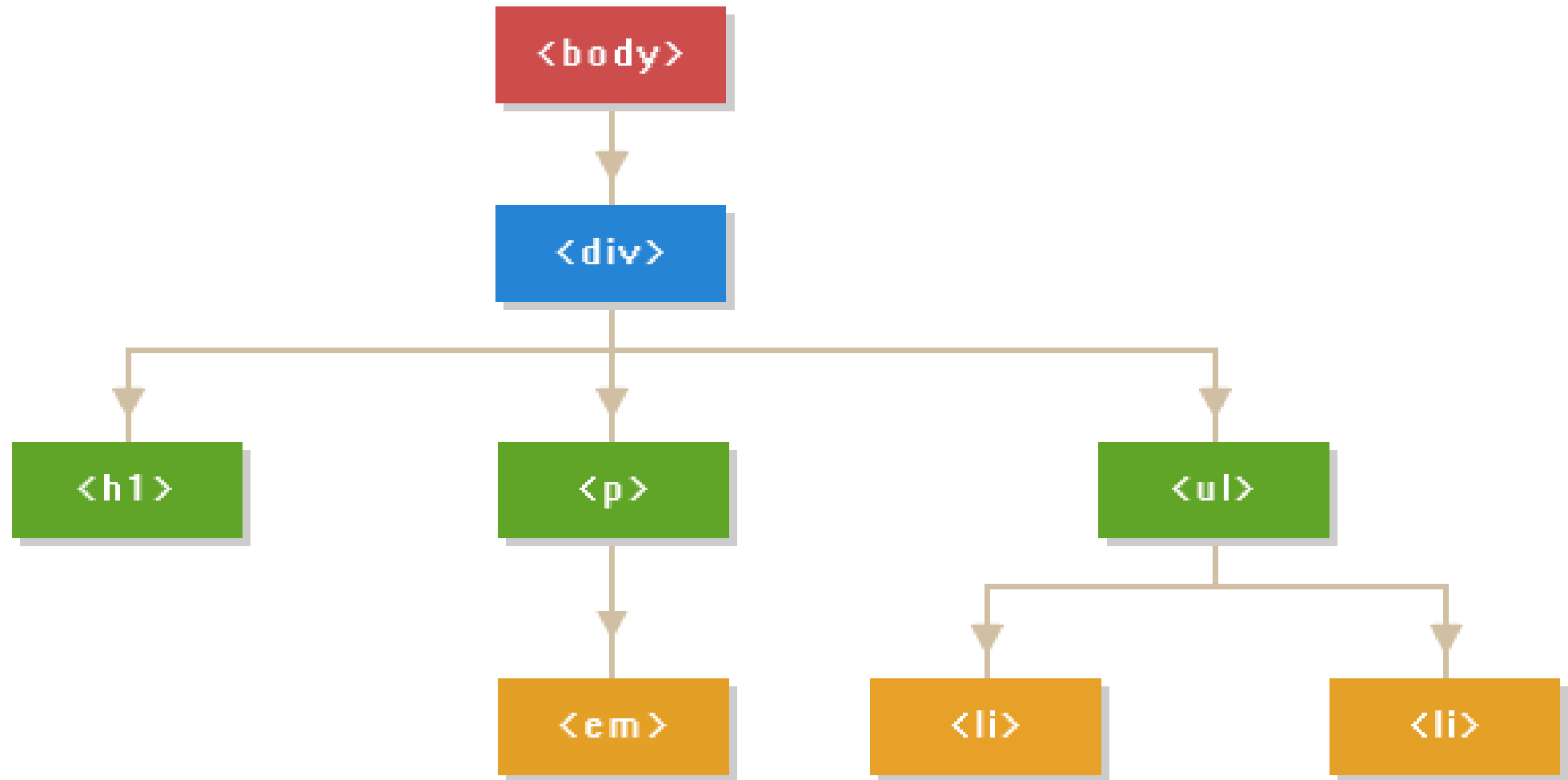

jQuery Traversing

- DOM traversing is one of the prominent features of the jQuery.
- To make the most it we need to understand the relationships between the elements in a DOM tree.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>HTML DOM Tree Sample</title>
<script src="jquery-1.12.4.min.js"></script>
</head>
<body>
  <div class="container">
    <h1>Hello World</h1>
    <p>This is a <em>simple paragraph</em>.</p>
    <ul>
      <li>Item One</li>
      <li>Item Two</li>
    </ul>
  </div>
</body>
</html>
```

The HTML code in the example above can be represented by the following DOM tree:



CMS[Joomla and Word Press]Overview

JOOMLA:

JOOMLA is an open source Content Management System(CMS),which allows user to build dynamic websites and blogs.

- Joomla! is a **Content Management System** (CMS) which enables you to build websites and powerful online applications.
- It is a simple and powerful web server application which requires a server with PHP and either MySQL to run. You can find full technical requirements here.
- Joomla! is **free and Open Source software** distributed under the GNU General Public License version 2 or later.

Some of the important features of Joomla are given below:

- Multilingual. ...
- Well-Supported. ...
- Easy Updates. ...
- Integrated Help System. ...
- Banner Management. ...
- Media Manager. ...
- Contact Management. ...
- Search.

- **User Manager** – It allows managing the user information such as permission to edit, access, publish, create or delete the user, change the password and languages. The main part of the user manager is *Authentication*.
- **Content Manager** – It allows managing the content using WYSIWYG editor to create or edit the content in a very simple way.
- **Banner Manager** – It is used to add or edit the banners on the website.
- **Template Manager** – It manages the designs that are used on the website. The templates can be implemented without changing the content structure within a few seconds.
- **Media Manager** – It is the tool for managing the media files and folder in which you can easily upload, organize and manage your media files into your article editor tool.
- **Contact Manager** – It allows to add contacts, managing the contact information of the particular users.
- **Web Link Manager** – The link resource is provided for user of the site and can be sorted into categories.
- **Search** – It allows users to search the appropriate information on the site. You can use smart indexing, advanced search options, auto suggest searches to make Joomla search best.
- **Menu Manager** – It allows to create menus and menu items and can be managed subsequently. You can put menu in any style and in multiple places.
- **RSS** – It stands for Really Simple syndication which helps your site contents and RSS files to be automatically updated.

Advantages of JOOMLA CMS

- It is an open source platform and available for free.
- Joomla is designed to be easy to install and set up even if you're not an advanced user.
- Since Joomla is so easy to use, as a web designer or developer, you can quickly build sites for your clients. With minimal instructions to the clients, clients can easily manage their sites on their own.
- It is very easy to edit the content as it uses WYSIWYG editor (**What You See Is What You Get** is a user interface that allows the user to directly manipulate the layout of the document without having a layout command).
- It ensures the safety of data content and doesn't allow anyone to edit the data.
- By default, Joomla is compatible with all browsers.
- The templates are very flexible to use.
- Media files can be uploaded easily in the article editor tool.
- Provides easy menu creation tool.

Disadvantages

- It gives compatibility problem while installing several modules, extensions and plugins simultaneously.
- Plugins and modules are not free in Joomla.
- Development is too difficult to handle when you want to change the layout.
- Joomla is not much SEO (Search Engine Optimization) friendly.
- It makes website heavy to load and run. Disadvantages
- It gives compatibility problem while installing several modules, extensions and plugins simultaneously.
- Plugins and modules are not free in Joomla.
- Development is too difficult to handle when you want to change the layout.
- Joomla is not much SEO (Search Engine Optimization) friendly.
- It makes website heavy to load and run.

Real World Examples of What Joomla Can Create?

- Corporate web sites or portals
- Corporate intranets and extranets
- Online magazines, newspapers, and publications
- E-commerce and online reservations
- Government applications
- Small business web sites
- Non-profit and organizational web sites
- Community-based portals
- School and religious web sites
- Personal or family homepages

Looking for an installable package?

- Joomla is not installable out of the box from this repository, please use:
- For the latest stable package: <https://downloads.joomla.org>
- For a nightly package: <https://developer.joomla.org/nightly-builds.html>

How to get a working installation from the source?

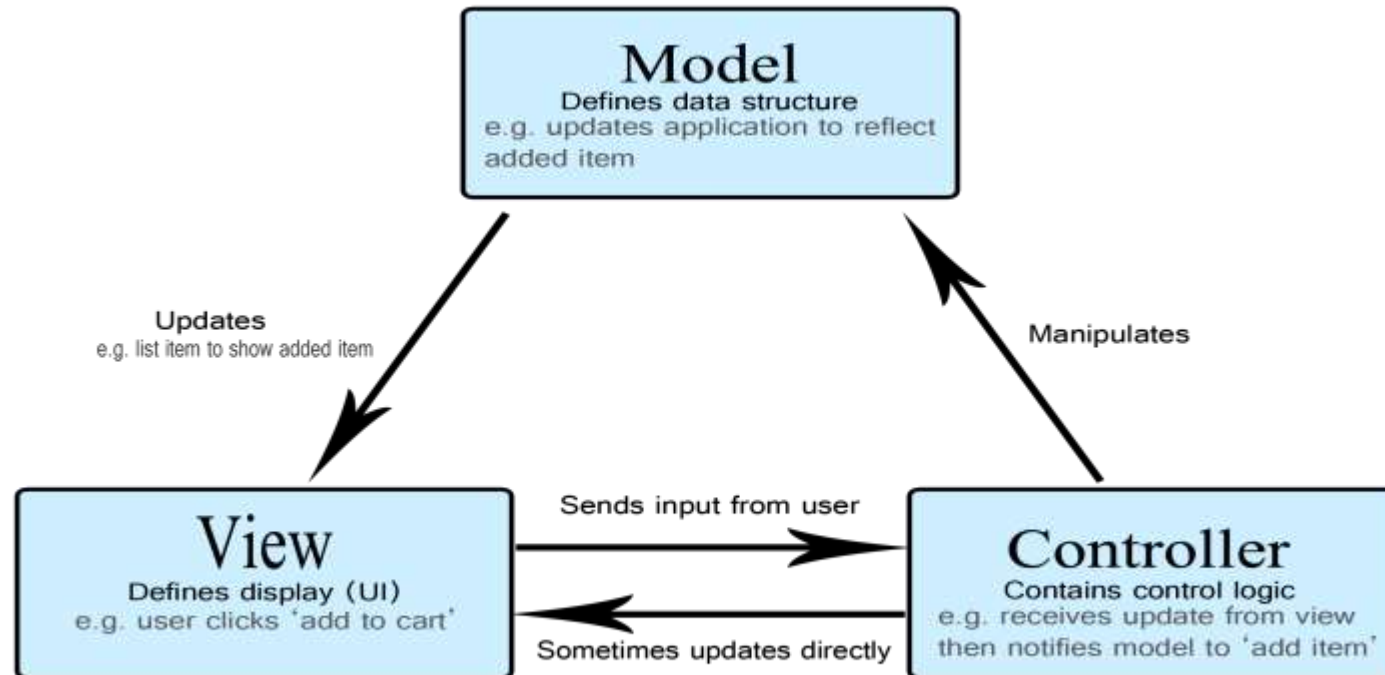
- For detailed instructions please visit https://docs.joomla.org/Special:MyLanguage/J4.x:Setting_Up_Your_Local_Environment
- You will need:
- PHP - basically the same as you need for running a Joomla Site, but you need the cli (command line interface) Version
(see https://docs.joomla.org/Special:MyLanguage/Configuring_a_LAMPP_server_for_PHP_development)
- Composer - for managing Joomla's PHP Dependencies. For help installing composer please read the documentation at <https://getcomposer.org/doc/00-intro.md>
- Node.js - for compiling Joomla's Javascript and SASS files. For help installing Node.js please follow the instructions available on <https://nodejs.org/en/>
- Git - for version management. Download from here <https://git-scm.com/downloads> (MacOS users can also use Brew and Linux users can use the built-in package manager, eg apt, yum, etc).

Creating Templates in JOOMLA

- Following are the simple steps to create templates in Joomla –
- **Step 1** – Create a folder called **MyFirstTemplate** inside your **Joomla → Templates** folder. Inside **MyFirstTemplate** folder, create 2 more folders named as *images* and *CSS* to save all the images and CSS files.
- **Step 2** – Inside **MyFirstTemplate** folder, create a file called as **templateDetails.xml**, without this file the template will not get displayed in Joomla

MVC Model for CMS

The **Model-View-Controller (MVC)** is an architectural pattern that separates an application into three main logical components: the **model**, the view, and the controller. Each of these components are built to handle specific development aspects of an application. MVC is one of the most frequently used industry-standard web development framework to create scalable and extensible projects. The following diagram shows the concept of MVC model.



CMS[Word Press]Overview

Word Press is an open source Content Management System(CMS),which allows user to build dynamic websites and blogs. Word press is the most popular blogging system on the web and allows updating, customizing and managing the website from its back-end CMS and components.

CMS[Content Management System]

The Content Management System(CMS) is a software which stores all the data such as text, photos, music, document etc. and is made available on website. It helps in editing, publishing and modifying the content of the website.

Features of CMS:

- User Management
- Media Management
- Theme System
- Plugins
- Mange over content
- Provide the template to user

Advantages

- It is an open source platform and available for free.
- CSS files can be modified according to the design as per users need.
- There are many plugins and templates available for free. Users can customize the various plugins as per their need.
- It is very easy to edit the content as it uses different editor .
- Media files can be uploaded easily and quickly. It offers several SEO tools which makes on site SEO simple.
- Customization is easy according to the user's needs.
- It allows creating different roles for users for website such as admin, author, editor and contributor.

Disadvantages

- Using several plugins can make the website heavy to load and run.
- PHP knowledge is required to make modifications or changes in the Word Press website.
- Sometimes software needs to be updated to keep the Word Press up-to-date with the current browsers and mobile devices.
- Modifying and formatting the graphic images and tables is difficult.

Model:-

The Model component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data. For example, a Customer object will retrieve the customer information from the database, manipulate it and update it data back to the database or use it to render data.

View:-

The View component is used for all the UI logic of the application. For example, the Customer view will include all the UI components such as text boxes, dropdowns, etc. that the final user interacts with.

Controller:-

Controllers act as an interface between Model and View components to process all the business logic and incoming requests, manipulate data using the Model component and interact with the Views to render the final output. For example, the Customer controller will handle all the interactions and inputs from the Customer View and update the database using the Customer Model. The same controller will be used to view the Customer data.